

My AI Engine

Motor personal de pensamiento para uso en Inteligencia Artificial — Manual de uso v2.0

autor: riv4wi — Mayo 2026

1. Qué es esto

Este motor es un framework minimalista de archivos de texto que actúan como **sistema operativo** del asistente de IA. En lugar de explicar cada vez tus reglas, estilo y arquitectura, le entregás los archivos al inicio de la sesión y la IA arranca configurada.

La filosofía central es la **separación entre el motor y los proyectos**: este repositorio contiene únicamente los 3 archivos núcleo genéricos y reutilizables. Los archivos específicos de cada proyecto viven dentro del repositorio de código de cada proyecto individual, no aquí.

Beneficios

- **Cero repetición**: tus reglas viven en archivos, no en la memoria de un chat.
- **Portabilidad**: funciona en Claude, ChatGPT, Gemini, DeepSeek y modelos locales.
- **Token-eficiente**: el SoT bajo trigger evita planes largos en tareas triviales.
- **Trazable**: cada decisión del agente puede rastrearse a una regla declarada.
- **Escalable**: cada proyecto instancia sus propias reglas sin contaminar el motor central.

2. Los 3 archivos núcleo

El motor consta de exactamente 3 componentes genéricos y reutilizables en cualquier proyecto.

Archivo	Tipo	Descripción
master_prompt.md	Universal	Núcleo inmutable. Instrucciones base, filosofía, formato y reglas de seguridad. Se carga en todas las sesiones.
preset_stack.md	Plantilla de stack	Plantilla base para crear módulos de tecnología por proyecto (ej: preset_laravel.md, preset_python.md).
contexto_proyecto.template.md	Plantilla de proyecto	Se instancia una vez por proyecto con stack, rutas, arquitectura y antipatrones. Vive en el repo del proyecto.

Regla mental: **master** → **preset** → **contexto**. Los tres se cargan juntos. El preset y el contexto vienen del repo del proyecto. Solo el master viene de my-ai-engine.

Ejemplo de flujo de trabajo en un proyecto real

Supongamos que trabajás en **apibml** con **Laravel** y **Oracle**:

Paso 1 — Preparación (una sola vez)

- Copiás preset_stack.md al proyecto como preset_laravel.md y lo completás.
- Copiás contexto_proyecto.template.md al proyecto como contexto_apibml.md.
- Estos archivos se versionan en el repo del proyecto.

Paso 2 — Carga en una sesión diaria

1. ~/my-ai-engine/master_prompt.md (el motor)
2. ~/proyectos/apibml/my-ai-engine/preset_laravel.md (reglas del stack)
3. ~/proyectos/apibml/my-ai-engine/contexto_apibml.md (estado del proyecto)

Para una sesión genérica: solo master_prompt.md.

3. Instalación

No hay instalación en el sentido tradicional. Los archivos viven en una carpeta de tu elección.

3.1. Estructura del repositorio my-ai-engine

```
~/my-ai-engine/
├─ README.md
├─ master_prompt.md
├─ preset_stack.md
├─ contexto_proyecto.template.md
└─ docs/
    ├─ MANUAL_motor_personal.pdf
    ├─ protocolo_pruebas_opencode.md
    └─ test-kit/
        ├─ contexto_task-api.md
        └─ task-api_preset_laravel.md
```

Los presets y contextos instanciados viven en el repo de cada proyecto:

```
~/proyectos/apibml/
├─ app/
└─ my-ai-engine/
    ├─ preset_laravel.md
    ├─ contexto_apibml.md
    └─ docs/
```

3.2. Pasos

- Descomprimir el ZIP en **~/my-ai-engine/**.
- Versionarlo en Git como repo personal. No agregar contextos de proyecto con datos sensibles.
- Por cada proyecto nuevo: copiar las plantillas al repo del proyecto y rellenarlas.
- Agregar al `.gitignore` los archivos con datos en zona roja/amarilla.

3.3. Aliases de ejemplo (bash / zsh)

```
alias motor-apibml='cat ~/my-ai-engine/master_prompt.md \
~/proyectos/apibml/my-ai-engine/preset_laravel.md \
~/proyectos/apibml/my-ai-engine/contexto_apibml.md | clip.exe'
```

En Linux/Mac reemplazar clip.exe por xclip -selection clipboard o pbcopy.

3.4. Uso con OpenCode Desktop (WSL2)

OpenCode lee AGENTS.md en la raíz del proyecto al iniciar la sesión. Generarlo concatenando los 3 archivos:

```
cat ~/my-ai-engine/master_prompt.md \
~/proyectos/mi-proyecto/my-ai-engine/preset_laravel.md \
~/proyectos/mi-proyecto/my-ai-engine/contexto_mi-proyecto.md \
> ~/proyectos/mi-proyecto/AGENTS.md
```

⚠ El agente NO detecta cambios automáticamente

AGENTS.md es estático — OpenCode lo lee solo al iniciar la sesión.

Cada vez que modifiques un archivo del motor:

1. `cp /tmp/master_prompt.md ~/my-ai-engine/master_prompt.md`

2. Regenerar AGENTS.md con el comando cat.
3. Abrir nueva sesión en OpenCode (no continuar la anterior).

4. Uso por plataforma de IA

4.1. Claude (web / desktop)

- Crear un **Project** por proyecto técnico y subir los 3 archivos como **knowledge files**.
- Para sesiones puntuales: pegar el contenido en un bloque <contexto>...</contexto>.

4.2. ChatGPT

- Crear un **Custom GPT** por proyecto y pegar los 3 archivos en las instrucciones del sistema.

4.3. Gemini

- Usar **Gems** para cargar el motor de forma persistente.
- Para 1M tokens: cargar los 3 archivos + código del proyecto vía Repomix.

4.4. Modelos locales (Ollama, Cline, OpenCode, etc.)

- Generar AGENTS.md o configurar como system prompt según la herramienta.
- Preferir modelos con $\geq 32k$ contexto para cargas grandes (Qwen2.5-Coder 32B, DeepSeek-Coder-V2).

5. Workflow por sesión

El flujo recomendado es siempre el mismo, independiente del proyecto:

Pa so	Acción	Resultado esperado
1	Cargar master + preset + contexto del proyecto.	La IA confirma reglas activas en una línea.
2	Describir la tarea.	La IA lista todo lo que falta en un único mensaje y espera respuesta.
3	Si es compleja, escribir modo SoT o /sot .	La IA responde con bloque SKETCH.
4	Aprobar o corregir el SKETCH.	La IA procede solo con OK explícito.
5	Recibir entregable.	Cierre con resumen en ≤ 3 viñetas.

6. El trigger SoT

El protocolo Sketch-of-Thought se activa solo con estos triggers: modo SoT, /sot, o «haz un sketch primero».

```
== SKETCH ==
[META]   : objetivo en una frase
[LOGIC]  : flujo en pseudocódigo
[RESTR]  : restricciones / dependencias
[RIESGO] : punto donde puede romperse

== BREAKPOINT ==
Si la tarea es grande: la IA se detiene y espera tu OK.

== SOLUCIÓN ==
```

[código / output ejecutable]

*Cuándo activarlo: refactors multi-archivo, arquitectura, migraciones, queries complejas.
Cuándo NO: dudas puntuales, syntax, debugging específico.*

7. Reglas de oro (resumen ejecutivo)

Las reglas declaradas en **master_prompt.md**. Si un asistente las viola, decirle «violaste la regla N» suele ser suficiente.

#	Regla
1	Nunca supongas. Identificá TODO lo que falta y preguntalo en UN SOLO mensaje antes del plan.
2	Plan → aprobación → ejecución. Nunca crear o modificar archivos sin plan y OK explícito.
3	Resuelve el error exacto del log primero.
4	Respetar lo que ya te di (no repreguntar).
5	Código mínimo verificable, nunca metadata. Mostrar siempre el código necesario para verificar — nunca descripciones o referencias a números de línea.
6	Mínima intervención, cero refactor preventivo.
7	Explica el «por qué», no solo el «cómo».
8	Documentación del proyecto = fuente de verdad.
9	Trazabilidad paso a paso en transformaciones.
10	Verifica entorno real antes de prescribir.
11	Verifica existencia antes de invocar.
12	Vista pre-armada antes que join crudo.

8. Mantenimiento del motor

8.1. Cuándo actualizar

- Corrección recurrente genérica → master_prompt.md.
- Corrección recurrente de stack → preset del proyecto.
- Proyecto cambia de stack → actualizar contexto en el repo del proyecto.
- Stack nuevo → crear preset_nuevo.md desde preset_stack.md.

8.2. Flujo de actualización en OpenCode (WSL2)

```
# 1. Copiar archivo modificado
cp /tmp/master_prompt.md ~/my-ai-engine/master_prompt.md

# 2. Regenerar AGENTS.md
cat ~/my-ai-engine/master_prompt.md \
  ~/proyectos/mi-proyecto/my-ai-engine/preset_laravel.md \
  ~/proyectos/mi-proyecto/my-ai-engine/contexto_mi-proyecto.md \
  > ~/proyectos/mi-proyecto/AGENTS.md

# 3. Abrir nueva sesión en OpenCode
```

8.3. Qué NO hacer

- No meter reglas de stack en master_prompt.md.
- No guardar contextos de proyecto en my-ai-engine/.

- No versionar contextos con datos reales en repos compartidos.
- No sobrecargar el master — si pasa 2 veces en un proyecto va al preset; si pasa en todos va al master.

8.4. Auditoría periódica

Cada ~3 meses: «audita nuestras conversaciones del último trimestre y dame las correcciones recurrentes». Las genéricas suben al master, las de stack al preset, las obsoletas bajan.

9. Roadmap — presets a crear por proyecto

Preset	Cuándo crearlo
preset_oracle.md	Bind variables OCI8, cursores, secuencias, restricciones 10g.
preset_python.md	FastAPI / Django. Tipado, async, manejo de excepciones.
preset_bash.md	Convenciones para scripts de automatización.
preset_react.md	Componentes, hooks, manejo de estado.

10. Filosofía del motor

Este sistema no es para que la IA sea más lista. Es para que **vos** dejes de repetir lo mismo en cada chat. La IA solo refleja lo que le diste; si no le diste reglas, te devuelve genéricos.

El valor del motor crece con el uso: cada corrección que codifiques en un archivo es una corrección que no tendrás que hacer nunca más. Tratalo como código: versionalo, refactorízalo, mantenelo limpio.

11. Limitaciones conocidas

11.1. No idempotencia garantizada

El motor reduce la probabilidad de comportamientos incorrectos mediante instrucciones explícitas, pero **no puede garantizar idempotencia absoluta con ningún LLM**. Los modelos de lenguaje son sistemas probabilísticos por diseño — la misma instrucción puede producir resultados diferentes entre ejecuciones.

- La diferencia entre modelos gratuitos y de pago es de **grado**, no de naturaleza.
- La confiabilidad varía según el modelo, la sesión y el contexto acumulado.
- Para uso en producción: validar el comportamiento con el modelo específico antes de confiar en él para tareas críticas.

11.2. No hay garantía de uso de la base de conocimientos

No existe forma técnica de garantizar que el modelo razonó usando el motor cargado. Estrategias para maximizar la verificabilidad:

Estrategia	Descripción	Limitación
Interrogatorio (T1)	Pedir que cite las reglas aplicadas al inicio de sesión.	Puede citar correctamente sin haberlas aplicado.

Estrategia	Descripción	Limitación
Trazabilidad en output	Referenciar la regla en cada decisión del plan.	Aumenta tokens; el modelo puede fabricar referencias.
Eval automatizado	Casos de prueba periódicos (como T1-T7).	Requiere mantenimiento; no cubre casos nuevos.

La combinación más robusta: **trazabilidad en output + eval automatizado periódico**.

11.3. Comportamiento dependiente del modelo

El motor fue validado con Big Pickle (OpenCode Desktop 1.15.5). Al cambiar de modelo, ejecutar el set T1-T7 completo antes de usar en producción.

Documento generado a partir de la auditoría cruzada de Gemini, ChatGPT, DeepSeek y Claude — mayo 2026. v2.0: estructura simplificada a 3 pilares. Reglas 1, 2 y 5 actualizadas tras protocolo de pruebas en OpenCode Desktop 1.15.5.